Structs, Méthodes et Composition

---

1. Les Structures (`struct`) et l'organisation mémoire

En Go, les classes (`class`) n'existent pas. Le seul moyen d'agréger des données composites est la **`struct`**. Cependant, Go enrichit ce concept avec des garanties d'initialisation, des métadonnées (*Struct Tags*), des méthodes rattachées et un mécanisme de composition native.

1.1 Déclaration, instanciation et *Zero Value* d'une `struct`

Lorsqu'une `struct` est déclarée sans initialisation explicite, **elle n'est jamais `nil`** : chacun de ses champs est automatiquement initialisé à sa propre *Zero Value*.

```Go
package main

import "fmt"

type Serveur struct {
	Hote     string
	Port     uint16
	Actif    bool
	ConnMax  int
}

func main() {
	// 1. Déclaration à sa Zero Value : Hote="", Port=0, Actif=false, ConnMax=0
	var s1 Serveur

	// 2. Littéral avec noms de champs (Recommandé : l'ordre n'a pas
d'importance
	// et les champs omis prennent leur Zero Value automatiquement)
	s2 := Serveur{
		Hote:  "10.0.0.1",
		Port:  8080,
		Actif: true,
	}

	// 3. Instanciation directe par pointeur (&T{}) — idiome standard en Go
	// (Remplace totalement malloc() + initialisation manuelle du C)
	ptrServeur := &Serveur{
		Hote: "192.168.1.50",
```

```
		Port: 443,
	}

	// En Go, l'opérateur flèche '->' du C n'existe pas !
	// On utilise le point '.' aussi bien sur une valeur que sur un pointeur
(déréférencement implicite) :
	ptrServeur.Actif = true // Équivalent de (*ptrServeur).Actif = true
	fmt.Println(s1, s2, ptrServeur.Hote)
}
```

1.2 Le pattern Constructeur (`New...`)

Go n'a pas de constructeurs syntaxiques. Lorsqu'une structure nécessite des valeurs par défaut ou une validation métier à la création, la convention universelle en Go consiste à exposer une fonction nommée `New<NomStruct>` (ou simplement `New` si elle est seule dans son package) :

```
Go
func NewServeur(hote string, port uint16) *Serveur {
	if port == 0 {
		port = 8080 // Valeur par défaut
	}
	return &Serveur{
		Hote:    hote,
		Port:    port,
		Actif:   true,
		ConnMax: 1000,
	}
}
```

1.3 Alignement mémoire et *Struct Padding* (Vue bas niveau / C)

Exactement comme un compilateur C, le compilateur Go aligne les champs d'une `struct` en mémoire sur des multiples de la taille des mots du processeur (8 octets sur architecture 64 bits) en insérant des octets de bourrage invisibles (*padding*).

```
Go
// MAUVAIS AGENCEMENT : 24 octets en mémoire !
type MauvaisAlignement struct {
	Actif    bool  // 1 octet  + 7 octets de padding !
	Compteur int64 // 8 octets
	Drapeau  bool  // 1 octet  + 7 octets de padding !
}
```

```
// BON AGENCEMENT (Du plus grand au plus petit) : 16 octets en mémoire (gain de
33% de RAM) !
type BonAlignement struct {
	Compteur int64 // 8 octets
	Actif    bool  // 1 octet
	Drapeau  bool  // 1 octet  + 6 octets de padding
}
```

Astuce : Vous pouvez inspecter la taille exacte et l'offset de chaque champ avec `unsafe.Sizeof(s)` et `unsafe.Offsetof(s.Champ)` du package `"unsafe"`.

1.4 La structure vide `struct{}` (0 octet) et les *Struct Tags*

- **`struct{}` (Empty Struct) :** Une structure sans aucun champ occupe **exactement 0 octet** en mémoire (`unsafe.Sizeof(struct{}{}) == 0`). Elle est utilisée partout en Go pour les ensembles (*Sets* `map[string]struct{}`) ou les signaux de notification dans les channels (`chan struct{}`).
- **Les *Struct Tags* :** Ce sont des chaînes de métadonnées brutes (placées entre backticks `` `...` ``) accolées aux champs d'une structure. Elles sont lues à l'exécution (via la réflexion) par les encodeurs JSON, XML, YAML ou les ORM SQL :

```
Go
type Utilisateur struct {
	ID       int    `json:"id" db:"user_id"`
	Nom      string `json:"nom_complet"`
	MotPasse string `json:"-"`                    // "-" : Toujours ignoré par
l'encodeur JSON !
	Email    string `json:"email,omitempty"`      // "omitempty" : Omis si le
champ est à sa Zero Value ("")
}
```

---

2. Les Méthodes et le choix du *Receiver*

En Go, on ne déclare pas les méthodes à l'intérieur des accolades de la `struct`. Une méthode est simplement une fonction ordinaire à laquelle on ajoute un paramètre spécial appelé **Receiver** (récepteur), placé juste entre le mot-clé `func` et le nom de la méthode.

Note : On peut attacher des méthodes à **n'importe quel type nommé** défini dans le même package (une `struct`, mais aussi un type basé sur `int`, `string` ou une `slice` !).

2.1 *Value Receiver* `(s Serveur)` vs *Pointer Receiver* `(s *Serveur)`

C'est l'une des distinctions les plus importantes en Go :

```go
type Compteur struct {
	Valeur int
}

// 1. VALUE RECEIVER : travaille sur une COPIE complète de la structure
func (c Compteur) IncrementerCopie() {
	c.Valeur++ // Modifie la copie locale : l'original reste intact !
}

// 2. POINTER RECEIVER : travaille sur l'adresse mémoire de la structure originale
func (c *Compteur) IncrementerReel() {
	c.Valeur++ // Modifie directement la structure originale en mémoire !
}
```

Sucre syntaxique du compilateur Go

En C, si une fonction attend `void inc(Compteur *c)`, vous devez écrire `inc(&monCompteur)`. En Go, si vous appelez `monCompteur.IncrementerReel()` sur une variable valeur, **le compilateur transforme automatiquement l'appel en `(&monCompteur).IncrementerReel()`** (et inversement, il déréférence automatiquement un pointeur si la méthode attend une valeur).

Les 3 règles d'or pour choisir entre Value et Pointer Receiver :

1. **Mutation :** La méthode doit-elle modifier l'état interne de la structure ? Si **OUI** → **Pointer Receiver (`*T`) obligatoire**.
2. **Performance / Copie :** La structure est-elle volumineuse (ex: plus de 64 octets ou nombreux champs) ou contient-elle un verrou de synchronisation (`sync.Mutex`, qui ne doit **jamais** être copié) ? Si **OUI** → **Pointer Receiver (`*T`) obligatoire**.
3. **Cohérence (Règle de style officielle Go) :** Si **une seule** méthode d'une structure a besoin d'un *Pointer Receiver*, alors **toutes** les méthodes de cette structure devraient utiliser un *Pointer Receiver* par cohérence (notamment pour la satisfaction des interfaces que nous verrons en J2S2).

2.2 Particularité : Appeler une méthode sur un pointeur `nil` !

En Java ou C++, appeler `objet.methode()` quand `objet` est `null`/`nullptr` provoque un crash immédiat avant même d'entrer dans la méthode. En Go, comme un *Pointer Receiver* `(c *Compteur)` n'est qu'un premier argument déguisé (`func IncrementerReel(c`

`*Compteur`), **il est tout à fait possible d'appeler une méthode sur un pointeur `nil`** et de gérer ce cas gracieusement :

```go
func (c *Compteur) LireValeur() int {
	if c == nil {
		return 0 // Gestion défensive d'un pointeur nil sans crash !
	}
	return c.Valeur
}
```

---

3. Composition et *Struct Embedding* (Zéro Héritage)

Go rejette catégoriquement l'héritage de classes (`extends`, hiérarchies parents/enfants virtuelles) qui crée un couplage fort et rigide. À la place, Go applique le principe **"Privilégiez la composition sur l'héritage"** grâce au mécanisme d'**imbrication anonyme (*Struct Embedding*)**.

3.1 Promotion automatique des champs et des méthodes

Lorsqu'on insère un type dans une `struct` **sans lui donner de nom de champ** (champ anonyme), tous les champs et toutes les méthodes de ce type imbriqué sont automatiquement **promus** au niveau de la structure englobante !

```go
package main

import "fmt"

// Brique de base réutilisable
type Horodatage struct {
	CreeLe    string
	ModifieLe string
}

func (h *Horodatage) MarquerModification(date string) {
	h.ModifieLe = date
}

// Brique métier qui COMPOSE Horodatage par Embedding (pas de nom de champ !)
type DocumentFacture struct {
	Horodatage        // <-- Struct Embedding (Champ anonyme)
	Numero     string
	MontantHT  float64
```

```
}

func main() {
	facture := DocumentFacture{
		Horodatage: Horodatage{CreeLe: "2026-01-10"}, // Initialisation
explicite
		Numero:     "FAC-2026-001",
		MontantHT:  1500.0,
	}

	// PROMOTION DES CHAMPS : Accès direct comme si 'CreeLe' appartenait à
DocumentFacture !
	fmt.Println("Créée le :", facture.CreeLe) // (équivalent de
facture.Horodatage.CreeLe)

	// PROMOTION DES MÉTHODES : Appel direct de la méthode de Horodatage sur
facture !
	facture.MarquerModification("2026-01-15")
	fmt.Println("Modifiée le :", facture.ModifieLe)
}
```

3.2 Masquage (*Shadowing*) vs Polymorphisme d'héritage

Attention à ne pas confondre *Embedding* et héritage POO :

- Si `DocumentFacture` définit sa propre méthode `MarquerModification()`, celle-ci **masque** (*shadows*) celle de `Horodatage` lorsqu'on l'appelle via `facture.MarquerModification()`.
- La méthode originale reste toujours accessible explicitement via `facture.Horodatage.MarquerModification()`.
- Il n'y a **pas de `super`** ni de classe de base commune : une fonction qui attend un paramètre de type `Horodatage` **n'acceptera pas** un `DocumentFacture` (le polymorphisme en Go se fait exclusivement via les **Interfaces**, sujet de la séquence J2S2).